

Mecanismos para solucionar el error de data race (acceso a variables compartido):



1. #pragma omp critical

- **Què fa:**
Garanteix que només **un fil a la vegada** entra a la secció marcada com *critical*. És una exclusió mútua **general**.
- **Com funciona:**
Tot el codi dins del bloc es protegeix.

None

```
#pragma omp critical
{
    sum += 4.0/(1.0 + x*x);
}
```

-

- **Pros:** molt flexible (pots protegir qualsevol fragment de codi).
- **Contres:** té **força overhead**, perquè sempre implica bloqueig i desbloqueig → pot ser lent si molts fils volen entrar.



2. #pragma omp atomic

- **Què fa:**
Protegeix **només una operació d'actualització senzilla** sobre una variable compartida (per exemple: suma, resta, producte, assignació, etc.).
- **Com funciona:**
El compilador i el maquinari s'asseguren que la instrucció és atòmica (no pot ser interrompuda per un altre fil).

None

```
#pragma omp atomic
sum += 4.0/(1.0 + x*x);
```

-

- **Pros:** més ràpid que critical, perquè no hi ha bloqueig general, només assegura la instrucció concreta.
- **Contres:** només serveix per operacions simples (*no pots posar codi arbitrari dins*).

+ 3. Reduction

- **Què fa:**
Crea **una còpia privada de la variable per cada fil**. Cada fil acumula el seu resultat en la seva còpia, i al final OpenMP combina totes les còpies (amb l'operació indicada: +, *, max, etc.) en la variable global.
- **Com funciona:**

None

```
double sum = 0.0;

#pragma omp parallel for reduction(+:sum)
for (int i=0; i<N; i++) {
    double x = (i+0.5)/N;
    sum += 4.0/(1.0 + x*x);
}
```

-
- **Pros:**
 - Evita completament *data races*.
 - No hi ha bloquejos dins del bucle → molt eficient.
 - Escala millor amb molts fils.
- **Contres:**
 - Només serveix per operacions associatives/commutatives (suma, producte, min, max...).
 - Una mica menys flexible que critical.

v0 = secuencial

v1 = #pragma omp parallel

v2 = #pragma omp parallel private (i,x)

v3 = contador num_threads y la i incrementa segun num_threads, osea hace bien el paralelismo

v4 = asegura que no se corrompa variable sum, con #parallel omp critical (hace que se haga de 1 en 1 para no modificar mal la variable)

v5 = cambia critical por #parallel omp atomic (es una operacion de actualizacion sencilla de una variable compartida)

v6 = crea una variable sum_local y despues del loop suma esa variable local a sum, utilizando #parallel omp atomic para que funcione correctamente

v7 = vuelve a la v2 pero cambia esto `#pragma omp parallel private (i,x) reduction(+:sum)`, crea variable local para cada thread y cuando acaba lo acumula en la variable global
v8 = añade `#pragma omp barrier` despues del loop, que es un punto donde todos los threads se esperan en el mismo punto y ninguno continua hasta que todos estén listos

1. 1.HELLO.C, HACER QUE SALGA HELLO WORLD 4 VECES:

```
paco1114@boada-11:~/lab2/openmp/day1$ OMP_NUM_THREADS=4 ./1.hello
Hello world!
Hello world!
Hello world!
Hello world!
```

2. 2.HELLO.C HACER QUE SALGA HELLO WORLD EN ORDEN

```
#pragma omp parallel num_threads(8) private(id)
```

3.

Q1: ¿Qué devuelve `omp_get_num_threads()` dentro y fuera de una región paralela?

Fuera de cualquier región paralela → siempre devuelve **1**, porque en ese momento sólo hay el hilo maestro activo.

Ejemplo:

```
printf("Starting, I'm alone ... (%d thread)\n",
omp_get_num_threads());
// salida: 1
```

•

Dentro de una región paralela → devuelve el número de hilos que forman ese equipo de trabajo, es decir, el tamaño del *parallel region*.

Ejemplo:

```
#pragma omp parallel
printf("%d\n", omp_get_num_threads());
```

•

- Si `OMP_NUM_THREADS=8`, cada hilo imprime **8**.
- Si usas `num_threads(4)`, imprime **4**.

♦ Q2: ¿Cuáles son las dos alternativas para sobrescribir el valor de `OMP_NUM_THREADS`?

- `#pragma omp parallel num_threads(4)`

- `OMP_NUM_THREADS`, pero esa es la configuración base, no una sobrescritura en tiempo de ejecución.)

♦ Q3: ¿Cuál es la “vida útil” de cada forma de definir el número de hilos?

- `OMP_NUM_THREADS` (variable de entorno)
 - Se aplica por defecto a todas las regiones paralelas del programa.
 - Es global y dura todo el tiempo de ejecución del programa (salvo que se sobrescriba en código).
- `num_threads(n)` en un pragma
 - Solo afecta a esa región paralela concreta.
 - Cuando termina la región, la configuración ya no aplica.
- `omp_set_num_threads(n)`
 - Modifica el número de threads por defecto que se usará en las próximas regiones paralelas *sin* `num_threads(...)` explícito.
 - Dura hasta que se cambie otra vez con otra llamada a `omp_set_num_threads()` o hasta que termine el programa.

4.

shared → todos los hilos escriben en la misma `x` sin control → queda el valor del último thread

private → cada hilo tiene su propia copia, no afecta a la `x` global → 5.

firstprivate → cada hilo copia el valor inicial de `x`, pero tampoco devuelve → 5.

reduction(+:x) → cada hilo suma su parte y al final se acumula en la `x` global → `5 + (0+1+2+3) = 11`.

5. PARA QUE DE CORRECTO EL MAX-VALUE:

- HEMOS PUESTO UN **#PRAGMA OMP CRITICAL** DENTRO DEL BUCLE, ANTES DEL IF
- **#pragma omp parallel private(i) reduction(max:maxvalue)** esta es **MÁS EFICIENTE**

6. PARA QUE DE CORRECTO EL MAX-VALUE:

- **#pragma omp parallel private(i) reduction(+:countmax)**